

Manual de Skills — Diego / Ergania

Diego

July 8, 2026

CONTENTS

Manual de Skills — Diego

1. Skills hechas a medida para Ergania

abogado-del-diablo

arranque

fable-plan

fixer

2. Familia “Fable 5” (genéricas, a nivel usuario)

3. G Stack — las más útiles para tu día a día

Flujo diario (bugs, QA, ship)

Planeación y diseño

Monitoreo y performance

Utilidades

Seguridad y control de cambios

4. Skills nativas de Claude Code (Anthropic) más útiles

5. Chuleta rápida — “tengo esta situación → uso esta skill”

Manual de Skills — Diego

Referencia rápida de las skills disponibles en Claude Code: las que hicimos a medida para Ergania, la familia genérica de la que nacieron, y las de G Stack / nativas de Claude Code que más sirven para tu forma de trabajar.

1. Skills hechas a medida para Ergania

Viven en `career-ops-ui/.claude/skills/` (carpeta no versionada — `.claude/` está en `.gitignore`). Tienen prioridad sobre cualquier skill genérica del mismo nombre cuando trabajas dentro de este repo, porque están escritas contra el código, los archivos y las cicatrices reales del proyecto, no en abstracto.

ABOGADO-DEL-DIABLO

Se dispara con: “qué te parece”, “estoy pensando en”, “quiero agregar”, “tengo una idea”, “deberíamos”, “me tinca”, “critica esto”, “sé honesto” — cualquier propuesta de feature, integración o cambio de rumbo, ANTES de planear o escribir código.

No se usa para: bugs (→ `fixer`), preguntas técnicas puntuales, trabajo ya decidido.

Qué hace: actúa como socio técnico escéptico. Prohibido validar por cortesía (“buena idea”, “me encanta”). Sigue un protocolo fijo de 4 pasos:

1. **Steelman** — la mejor versión honesta de tu idea, más fuerte de como la contaste.
2. **Ataque** — cuatro preguntas ancladas a la realidad de Ergania: qué la haría fallar en un mes en una serverless function sin colas mantenida 2–5 h/semana; quién de los usuarios reales no la usaría; cuál es la alternativa más barata que logra el 80%; qué costo oculto trae (horas, plata de API, superficie de fallo, líneas al god-file `careersController.ts`).
3. **Riesgos rankeados** — tabla probabilidad × impacto, máximo 5.
4. **Veredicto obligatorio** — SEGUIR / CAMBIAR / MATAR, sin “depende”. Si es SEGUIR, trae los 3 cambios que más lo mejoran.

Mantiene una “ficha de realidad” con cifras del negocio (trial 3 días, \$9.990 CLP/mes, presupuesto de 2–5 h/semana, arquitectura serverless sin workers ni cron, 0 monitoreo en producción) para que cada crítica apunte a algo concreto, no a jerga de manual.

ARRANQUE

Se dispara con: “proyecto nuevo”, “nueva app”, “empecemos”, “MVP”, “idea nueva”, “scaffolding” — o cuando el directorio está vacío o el repo tiene 0–1 commits.

No se usa para: agregar features a un proyecto ya en marcha (→ `fable-plan`).

Qué hace: manual de arranque destilado de los dolores reales de Ergania (git que llegó con datos personales horneados en la historia, deploy tardío que cobró su deuda en una noche de 6 fixes, 4 pivotes de proveedor de IA, 6 enfoques distintos para generar un PDF). Regla madre: *una decisión reversible tomada ya vale más que la “perfecta” tomada después* — lo único irreversible es la historia de git y los datos que entren en ella.

Estructura en 4 fases:

- **Fase 0** — 3 preguntas obligatorias antes de codear: qué problema resuelve, quién lo usa (persona concreta), cuál es la primera cosa visible que demuestra que funciona. Van al CLAUDE.md del proyecto nuevo.
- **Fase 1** — decisiones de stack aburrido y documentado (stack default: React+Vite+Tailwind / Express+TS serverless / Supabase / Vercel / MercadoPago Checkout Pro), con el porqué anotado ANTES de codear: dónde corre en producción, qué proveedor de IA y quién paga, dónde se generan artefactos pesados.
- **Fase 2** — checklist del día 1 en orden: `git init` antes del primer archivo, `.gitignore` antes del primer commit (revisar archivo por archivo — sin datos personales), CLAUDE.md desde el día 1, un solo comando de desarrollo, validación de env vars al boot (BOM, `\r\n`, fail-fast), **deploy el día 1** aunque sea un “hola”, ramas + PR desde el día 1.
- **Fase 3** — reglas de crecimiento: cero sobre-ingeniería pero dividir archivos que pasan de ~400–500 líneas, cada bug de producción deja un test de regresión, cada gotcha va al CLAUDE.md el mismo día, desinstalar dependencias muertas en el mismo commit que se abandona el enfoque.

Termina con un checklist de salida — el arranque no se considera terminado hasta que todos los puntos son verdad.

FABLE - PLAN

Se dispara con: “agreguemos”, “quiero que la app haga X”, “cómo implementamos”, “planea”, “diseña esta función” — ANTES de escribir código.

No se usa para: bugs (→ `fixer`), proyectos desde cero (→ `arranque`), cambios triviales de una línea.

Qué hace: planifica features nuevas leyendo primero el código real de Ergania. Regla madre: *nunca proponer sin explorar* — un plan escrito antes de leer el código es ficción con formato de plan.

- **Fase 1** — tabla de qué leer según la zona que toca el cambio (UI de una página, endpoint nuevo, cambio de schema, feature con IA, auth/suscripciones, ruta nueva, landing/copy), más los patrones fijos del repo: flujo `routes → controller` (`try/catch`, error en español) → `service` (keyed por `user_email`), frontend que solo llama al backend vía `lib/api.ts`, todo corriendo en una sola serverless function.
- **Fase 2** — 6 preguntas obligatorias por escrito: problema real detrás del pedido, lo más pequeño que lo resuelve, qué se rompe (checklist específico: tablas nuevas que entran a backup/restore, multi-perfil, gating de suscripción, SSE con token por query param, archivos-dios que exigen cambios quirúrgicos), casos límite, cómo se verifica, qué NO se va a hacer.
- **Fase 3** — formato de plan liviano (Problema real / Lo mínimo / Decisiones tomadas / No haremos / pasos numerados, cada uno con su verificación), en rama `feature/<nombre>`, migraciones SQL aditivas primero.
- **Fase 4** — tabla de comandos de verificación reales del repo (tests frontend, builds de tipos, dev local, `/review`, `/qa`, ship a `master`).

FIXER

Se dispara con: “bug”, “error”, “falla”, “no funciona”, “se rompió”, “está roto”, “no carga”, “500”, “401”, “dejó de funcionar” — y también cuando otra sesión o modelo afirma que algo “ya quedó arreglado” y hay que verificarlo.

No se usa para: features nuevas (→ `fable-plan`), proyectos desde cero (→ `arranque`).

Qué hace: método de 6 pasos con evidencia obligatoria en cada uno — “debería funcionar” no cuenta:

1. **Reproducir primero** — nunca tocar código sin ver el bug pasar. Distingue el flujo según sea UI, API local, o solo-en-producción (ojo: `ergania.com` sin `www` responde 308 y puede romper clientes HTTP).
2. **Causa raíz, no síntoma** — mapa de dónde se decide cada tipo de estado (suscripción, auth/sesión, config de Supabase, rutas HTTP).
3. **Arreglo mínimo** — solo lo que la causa raíz exige, nada de refactors de paso.
4. **Probar con evidencia** — el caso exacto pasando, suite completa en verde, ambos typechecks en verde, más un test de regresión en `regressions.test.tsx` si el bug era de UI.
5. **La regla contra el “ya quedó”** — sin output que lo demuestre, se trata como NO arreglado. Un mensaje de commit “fix” no es evidencia.
6. **Reporte fiel** — si algo falla o se saltó un paso, se dice explícitamente.

Incluye la tabla completa de comandos de verificación del repo, dónde viven los logs (consola del navegador, stdout local, `vercel logs`, dashboard de Supabase) y los gotchas ya conocidos (redirect 308 a `www`, env vars por rama, `\r\n` pegado en `SUPABASE_URL`, allow-list de OAuth, sesión única por cuenta).

2. Familia “Fable 5” (genéricas, a nivel usuario)

Instaladas en `~/ .claude/skills/`, sirven para **cualquier proyecto**, no solo Ergania. Cuatro de ellas (`abogado-del-diablo`, `arranque`, `fable-plan`, `fixer`) ya tienen versión propia para Ergania arriba, que gana por ser más específica dentro de este repo. `fable-chequeo-seguridad` todavía no tiene versión Ergania — es la única de la familia que hoy solo existe en su forma genérica.

Skill	Para qué sirve
<code>fable-abogado-del-diablo</code>	Crítica constructiva de un plan o idea, estilo Fable 5, sin el contexto específico de Ergania. Útil en proyectos nuevos que aún no tienen su propia ficha de realidad.
<code>fable-arranque</code>	Igual que <code>arranque</code> , pero genérico — para arrancar cualquier proyecto nuevo sin las cicatrices particulares de Ergania.
<code>fable-chequeo-seguridad</code>	Chequeo de seguridad estilo Fable 5. Usar antes de publicar, o después de agregar logins, pagos o manejo de datos de usuarios — en cualquier proyecto.
<code>fable-fixer</code>	Igual que <code>fixer</code> , pero sin los comandos y gotchas específicos de este repo.
<code>fable-plan</code>	Versión genérica de planeación de features, estilo Fable 5.

3. G Stack — las más útiles para tu día a día

Instalado en `~/.claude/skills/gstack` (v1.58.5.0). Se invocan como `/nombre` .

FLUJO DIARIO (BUGS, QA, SHIP)

Skill	Para qué sirve
<code>/investigate</code>	Debugging sistemático con análisis de causa raíz — para bugs difíciles de reproducir.
<code>/qa</code>	QA completo de la app: build + pruebas de comportamiento end-to-end.
<code>/qa-only</code>	Igual que <code>/qa</code> pero sin el paso de build — más rápido cuando ya sabes que compila.
<code>/review</code>	Code review del diff actual antes de abrir PR.
<code>/ship</code>	Flujo completo: land + deploy con review incluido.
<code>/land-and-deploy</code>	Variante del flujo de deploy, útil si necesitas separar el paso de “aterrizar” del de “desplegar”.
<code>/health</code>	Dashboard de calidad del código del repo.

PLANEACIÓN Y DISEÑO

Skill	Para qué sirve
<code>/spec</code>	Convierte una intención vaga en un spec técnico preciso y ejecutable — útil para features grandes con incertidumbre de producto (alternativa pesada al plan liviano de <code>fable-plan</code>).
<code>/office-hours</code>	Sesión estilo YC Office Hours para pensar en voz alta sobre producto/estrategia.
<code>/design-consultation</code>	Consultoría de sistema de diseño completo (estética, tipografía, color, layout).
<code>/design-shotgun</code>	Genera múltiples variantes de UI para comparar.
<code>/design-html</code>	Genera HTML/CSS de producción a partir de un diseño ya decidido.
<code>/design-review</code>	Auditoría visual: detecta inconsistencias, problemas de espaciado y jerarquía.
<code>/plan-eng-review</code>	Revisión de un plan desde la óptica de ingeniería.
<code>/plan-design-review</code>	Revisión de un plan desde la óptica de diseño.
<code>/plan-ceo-review</code>	Revisión de un plan a nivel estrategia/negocio.
<code>/plan-devex-review</code>	Revisión de un plan desde experiencia de desarrollador.
<code>/autoplan</code>	Pipeline que corre las 4 revisiones anteriores en secuencia con decisiones automáticas.

MONITOREO Y PERFORMANCE

Skill	Para qué sirve
<code>/canary</code>	Monitoreo post-deploy — detecta si algo se rompió recién desplegado.
<code>/benchmark</code>	Detección de regresiones de performance.
<code>/retro</code>	Retrospectiva del trabajo reciente.

UTILIDADES

Skill	Para qué sirve
<code>/make-pdf</code>	Convierte markdown en PDF de calidad de publicación (con el que se generó este documento).
<code>/diagram</code>	Genera diagramas (mermaid/excalidraw) desde una descripción en inglés.
<code>/document-generate</code>	Genera documentación desde cero para una feature o módulo.
<code>/document-release</code>	Actualiza documentación después de un deploy.
<code>/context-save</code> / <code>/context-restore</code>	Guarda y restaura el contexto de trabajo entre sesiones.
<code>/learn</code>	Modo aprendizaje guiado sobre el codebase.
<code>/codex</code>	Wrapper del CLI de OpenAI Codex, para segunda opinión de otro modelo.
<code>/skillify</code>	Convierte un flujo de <code>/scrape</code> exitoso en una skill permanente.
<code>/gstack-upgrade</code>	Actualiza G Stack a la última versión.

SEGURIDAD Y CONTROL DE CAMBIOS

Skill	Para qué sirve
<code>/cso</code>	Modo Chief Security Officer — revisión de seguridad más profunda.
<code>/freeze</code>	Restringe las ediciones de Claude a un directorio específico durante la sesión.
<code>/guard</code>	Modo seguro completo: advertencias destructivas + ediciones restringidas por carpeta.
<code>/careful</code>	Solo las advertencias para comandos destructivos, sin restringir carpetas.

4. Skills nativas de Claude Code (Anthropic) más útiles

No son de G Stack — vienen incluidas en Claude Code y son intercambiables entre cualquier proyecto.

Skill	Para qué sirve
verify	Verifica que un cambio de código de verdad hace lo que debe, ejecutando el flujo afectado de punta a punta — no solo tests o typecheck.
code-review	Revisa el diff actual en busca de bugs de correctitud y limpiezas de reuso/simplicidad/eficiencia, con niveles de esfuerzo ajustables (bajo → ultra).
simplify	Revisa el código cambiado buscando solo simplificación y reuso — no bugs — y aplica los fixes directo.
security-review	Revisión de seguridad completa de los cambios pendientes en la rama actual.
run	Levanta y ejecuta la app para ver un cambio funcionando de verdad, no solo en tests.
schedule	Crea o actualiza tareas programadas (cron) que corren automáticamente.
consolidate-memory	Pasada reflexiva sobre los archivos de memoria: fusiona duplicados, corrige datos viejos, poda el índice.
claude-api	Referencia técnica de la API de Claude/Anthropic (modelos, precios, streaming, tool use, MCP).

5. Chuleta rápida — “tengo esta situación → uso esta

skill”

Situación	Skill
Tengo una idea/feature nueva y quiero saber si vale la pena	abogado-del-diablo
Voy a construir una feature ya decidida	fable-plan
Feature grande, con harta incertidumbre de producto	/spec
Voy a empezar un proyecto nuevo desde cero	arranque
Hay un bug y necesito arreglarlo	fixer
El bug es difícil de reproducir o entender	/investigate
Otra sesión dice “ya quedó arreglado” y no confío	fixer (Paso 5)
Quiero revisar mi diff antes de abrir PR	/review o code-review
Quiero limpiar código sin buscar bugs	simplify
Voy a hacer deploy	/ship
Acabo de deployar, quiero vigilar que no se rompió nada	/canary
Quiero ver variantes visuales de una pantalla	/design-shotgun
Quiero pulir el detalle visual de algo ya construido	/design-review
Voy a tocar auth, pagos o datos de usuarios	fable-chequeo-seguridad o security-review
Necesito un PDF de un documento markdown	/make-pdf
Necesito un diagrama de un flujo	/diagram
Quiero guardar dónde quedé para retomar después	/context-save
Quiero saber si el código realmente hace lo que debería, no solo si compila	verify